

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное
образовательное учреждение высшего образования «Тульский
государственный университет»
Институт прикладной математики и компьютерных наук

ЛАБОРАТОРНАЯ РАБОТА №7
по дисциплине
Интеграция и тестирование программных систем
по теме
«Интеграционное тестирование веб-приложения в Docker-окружении»

Выполнил: ст. гр. 220032-11

Лаукерт К.М.

Принял:

Басалов Ю.А.

Тула 2026

Лабораторная работа №7

по теме «Интеграционное тестирование веб-приложения в Docker-окружении»

Цель работы

Получить практические навыки интеграционного тестирования веб-приложения, развёрнутого в Docker-контейнерах, с проверкой HTTP API и фактического состояния базы данных PostgreSQL.

Выбранный проект

Для варианта 9 выбран открытый проект Node.js CRUD Backend: <https://github.com/EsankiThambawita/Dockerized-Node.js-Express-PostgreSQL-CRUD-Backend>. Приложение реализовано на Node.js и Express, использует драйвер pg и хранит данные в PostgreSQL. Объектом API является таблица schools с полями id, name и address.

Архитектура тестового окружения

```
pytest + requests -> HTTP :13000 -> Express (Node.js) ->
PostgreSQL :5433
```

```
psycopg2 -----^
```

Docker Compose запускает два сервиса: app — Express-приложение, опубликованное на порту 13000, и db — PostgreSQL 16 на порту 5433. Сервис app ожидает успешного healthcheck PostgreSQL. Тесты обращаются к API через requests, а затем подключаются к той же БД через psycopg2 для независимой валидации сохранённых данных.

Развёртывание Docker Compose

Исходный docker-compose.yaml был дополнен директивой build для приложения, публикацией порта PostgreSQL и healthcheck. Окружение запускается из каталога ЛР7/project командой docker compose up -d --build.

```
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_PASSWORD: password123
      POSTGRES_USER: user123
      POSTGRES_DB: db123
```

```

ports:
  - "5433:5432"
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U user123 -d db123"]
  interval: 5s
  timeout: 5s
  retries: 10
app:
  build: .
  ports:
    - "13000:3000"
  depends_on:
    db:
      condition: service_healthy

```

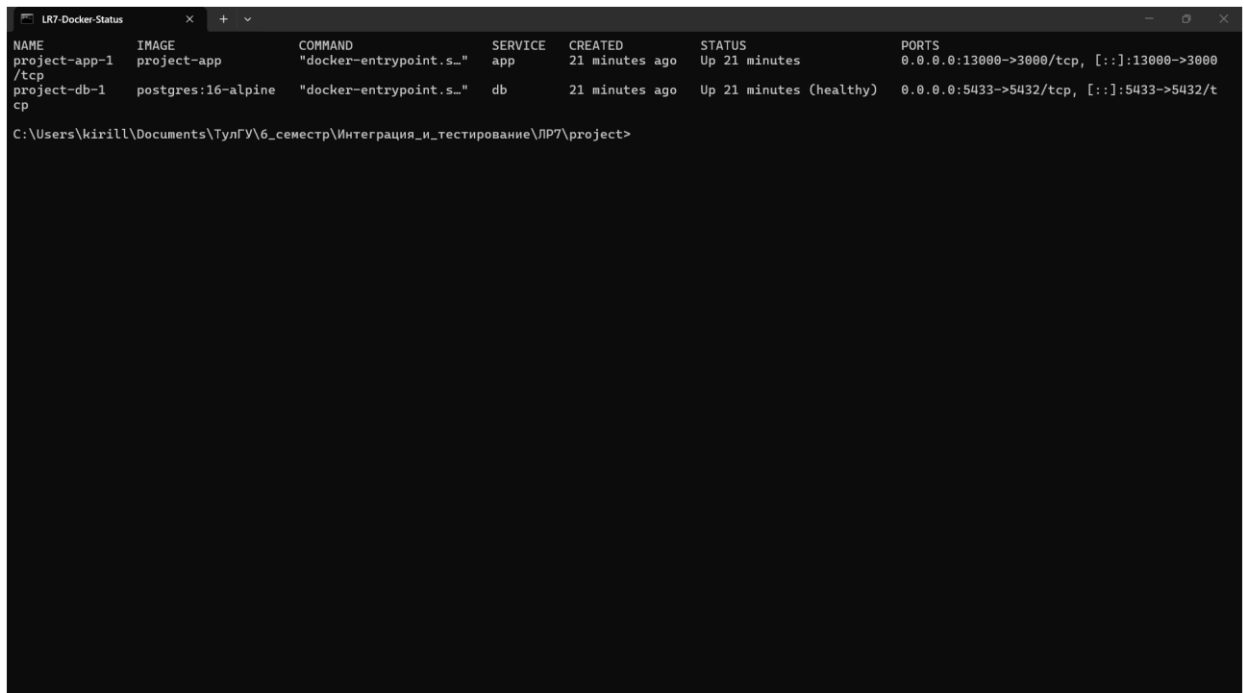


Рисунок 1 — запущенные контейнеры приложения и PostgreSQL; база данных имеет статус healthy

Структура тестового проекта

Файл или каталог	Назначение
integration_tests/conftest.py	Фикстуры API-сессии, подключения к PostgreSQL и очистки таблицы перед каждым тестом
integration_tests/test_schools_integration.py	Пять сквозных интеграционных сценариев API ↔ PostgreSQL
integration_tests/requirements.txt	Зависимости pytest, requests, psycopg2-binary и allure-pytest

integration_tests/allure-results/	Исходные данные Allure, созданные pytest
integration_tests/allure-report/	Сформированный HTML-отчёт Allure

Фикстуры pytest

Фикстура `api_session` ожидает доступности контейнеризированного API и инициализирует схему через GET `/setup`. Фикстура `db_connection` создаёт прямое подключение к PostgreSQL. Автоматическая фикстура `clean_schools` очищает таблицу перед каждым сценарием, благодаря чему тесты независимы.

```
"""Pytest fixtures for API-to-PostgreSQL integration tests."""
```

```
import time
```

```
import psycopg2
```

```
import pytest
```

```
import requests
```

```
API_URL = "http://127.0.0.1:13000"
```

```
DB_SETTINGS = {
    "host": "127.0.0.1",
    "port": 5433,
    "dbname": "db123",
    "user": "user123",
    "password": "password123",
}
```

```
@pytest.fixture(scope="session")
```

```
def api_session():
```

```
    """Return a session after the containerized API and its
    schema are ready."""
```

```
    session = requests.Session()
```

```
    deadline = time.monotonic() + 30
```

```
    while time.monotonic() < deadline:
```

```
        try:
```

```
            response = session.get(f"{API_URL}/setup",
```

```
            timeout=2)
```

```

        if response.status_code in (200, 500):
            return session
    except requests.RequestException:
        time.sleep(1)
    pytest.fail("Containerized API did not become ready within
30 seconds")

```

```

@pytest.fixture(scope="session")
def db_connection():
    connection = psycopg2.connect(**DB_SETTINGS)
    yield connection
    connection.close()

```

```

@pytest.fixture(autouse=True)
def clean_schools(db_connection, api_session):
    """Keep each scenario independent while preserving the
    schema."""
    with db_connection.cursor() as cursor:
        cursor.execute("TRUNCATE TABLE schools RESTART
IDENTITY")
    db_connection.commit()
    yield

```

```

@pytest.fixture
def create_school(api_session):
    def _create(name: str, location: str):
        return api_session.post(
            API_URL,
            json={"name": name, "location": location},
            timeout=5,
        )

    return _create

```

Интеграционные сценарии

Реализованы пять тест-кейсов: проверка создания таблицы schools; получение пустого списка из чистой БД; создание школы через POST и проверка строки SQL-запросом; получение ранее созданной школы через GET и проверка её полей в БД; создание двух школ последовательными HTTP-запросами и проверка всех строк PostgreSQL.

```
"""End-to-end checks for the Express API and PostgreSQL
persistence layer."""
```

```
import allure
```

```
from conftest import API_URL
```

```
@allure.feature("Schools API")
```

```
@allure.story("Schema initialization")
```

```
def test_setup_endpoint_created_schools_table(api_session,
db_connection):
```

```
    with allure.step("Verify the table created by GET /setup"):
```

```
        with db_connection.cursor() as cursor:
```

```
            cursor.execute("SELECT
```

```
to_regclass('public.schools')")
```

```
            assert cursor.fetchone()[0] == "schools"
```

```
@allure.feature("Schools API")
```

```
@allure.story("Read schools")
```

```
def test_get_returns_empty_list_for_clean_database(api_session,
db_connection):
```

```
    response = api_session.get(API_URL, timeout=5)
```

```
    assert response.status_code == 200
```

```
    assert response.json() == []
```

```
    with db_connection.cursor() as cursor:
```

```
        cursor.execute("SELECT COUNT(*) FROM schools")
```

```
        assert cursor.fetchone()[0] == 0
```

```

@allure.feature("Schools API")
@allure.story("Create school")
def test_post_persists_school_in_postgresql(create_school,
db_connection):
    response = create_school("School No. 9", "Tula")

    assert response.status_code == 200
    assert response.json() == {"message": "Successfully added
child"}
    with db_connection.cursor() as cursor:
        cursor.execute("SELECT name, address FROM schools")
        assert cursor.fetchone() == ("School No. 9", "Tula")


@allure.feature("Schools API")
@allure.story("Read after create")
def test_created_school_is_returned_by_get(create_school,
api_session, db_connection):
    create_school("Gymnasium", "Lenina 10")

    response = api_session.get(API_URL, timeout=5)

    assert response.status_code == 200
    assert response.json() == [{"id": 1, "name": "Gymnasium",
"address": "Lenina 10"}]
    with db_connection.cursor() as cursor:
        cursor.execute("SELECT id, name, address FROM schools")
        assert cursor.fetchone() == (1, "Gymnasium", "Lenina
10")


@allure.feature("Schools API")
@allure.story("Multiple writes")
def test_multiple_api_requests_persist_all_rows(create_school,
api_session, db_connection):
    assert create_school("Lyceum", "Sovetskaya 1").status_code
== 200

```

```

assert create_school("College", "Mira 2").status_code == 200

response = api_session.get(API_URL, timeout=5)

assert [school["name"] for school in response.json()] ==
["Lyceum", "College"]

with db_connection.cursor() as cursor:
    cursor.execute("SELECT name, address FROM schools ORDER
BY id")

assert cursor.fetchall() == [
    ("Lyceum", "Sovetskaya 1"),
    ("College", "Mira 2"),
]

```

Результат выполнения

```

===== test session starts =====
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\kirill\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
metadata: {'Python': '3.14.7', 'Platform': 'Windows-11-10.0.26200-SP0', 'Packages': {'pytest': '9.1.1', 'pluggy': '1.6.0'}, 'Plugins': {'allure-pytest': '2.16.2', 'Faker': '40.40.0', 'html': '4.2.0', 'metadata': '3.1.1', 'rerunfailures': '16.7', 'xdist': '3.8.0'}}
rootdir: C:\Users\kirill\Documents\ТунГУ\6_семестр\Интеграция_и_тестирование\ЛР7\integration_tests
plugins: allure-pytest-2.16.2, Faker-40.40.0, html-4.2.0, metadata-3.1.1, rerunfailures-16.7, xdist-3.8.0
collected 5 items

test_schools_integration.py::test_setup_endpoint_created_schools_table PASSED [ 20%]
test_schools_integration.py::test_get_returns_empty_list_for_clean_database PASSED [ 40%]
test_schools_integration.py::test_post_persists_school_in_postgresql PASSED [ 60%]
test_schools_integration.py::test_created_school_is_returned_by_get PASSED [ 80%]
test_schools_integration.py::test_multiple_api_requests_persist_all_rows PASSED [100%]

===== 5 passed in 1.58s =====
C:\Users\kirill\Documents\ТунГУ\6_семестр\Интеграция_и_тестирование\ЛР7\integration_tests>

```

Рисунок 2 — фактический запуск pytest: все 5 интеграционных тестов прошли

Тесты запущены командой `py -m pytest -v --alluredir=allure-results`. Все пять сценариев прошли успешно: 5 passed.

По результатам pytest сформирован Allure HTML-отчёт. Его итоговая страница показывает 5 тест-кейсов и успешность 100%.

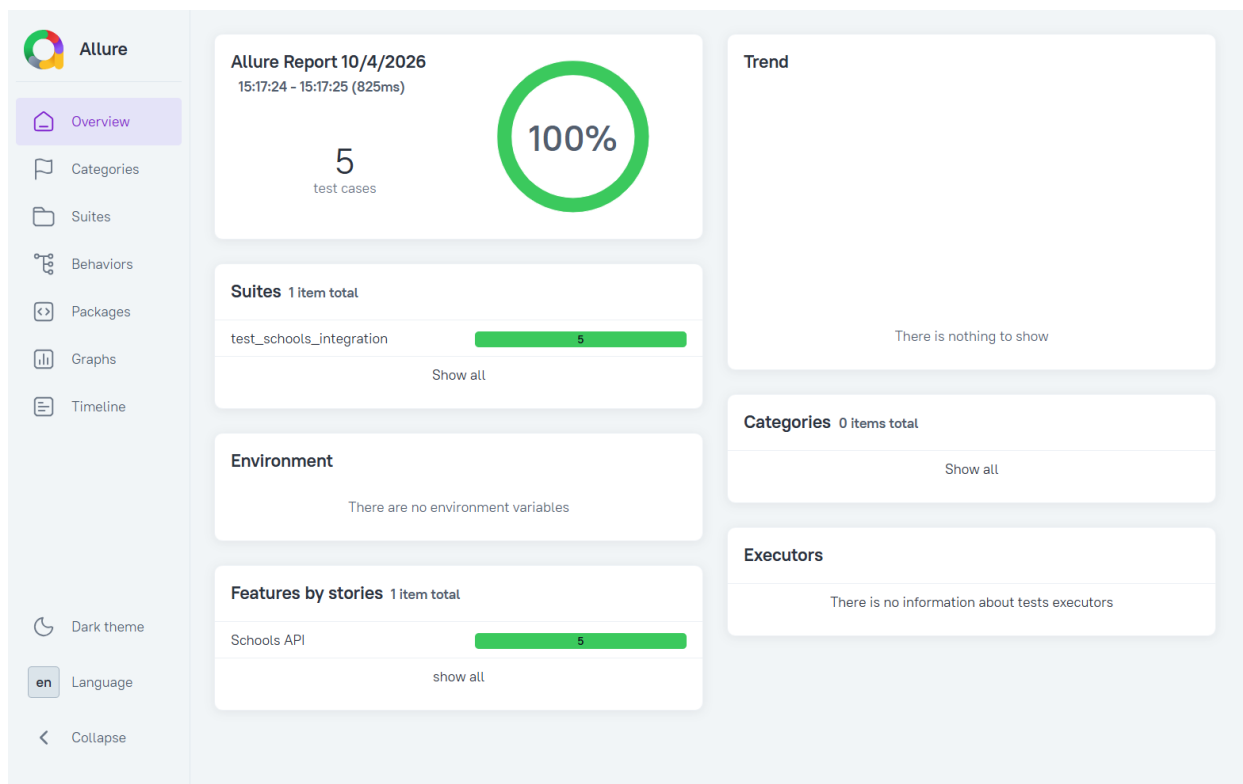


Рисунок 3 — итоговая страница Allure: 5 test cases, 100%

Выводы

В работе развёрнуто Docker-окружение Node.js-приложения и PostgreSQL, подготовлены изолированные интеграционные тесты pytest. Каждый сценарий проверяет не только HTTP-ответ API, но и состояние PostgreSQL через прямое SQL-подключение. Все пять реализованных сценариев успешно прошли; результаты сформированы в Allure-отчёте.